

BOOBUZZ architecture -- diagram-first

This document describes the checked-out **dev-phase-1.1** implementation. The requested R3 anchors are robot-code d4226eb and re-cock-nize 76db5f4; the current **dev-phase-1.1** heads used for this refresh are robot-code 907c31a and re-cock-nize 518bf9c. Source diagrams remain fenced here; **architecture/build.sh** renders them to **architecture/diagrams/** before Pandoc creates the PDF.

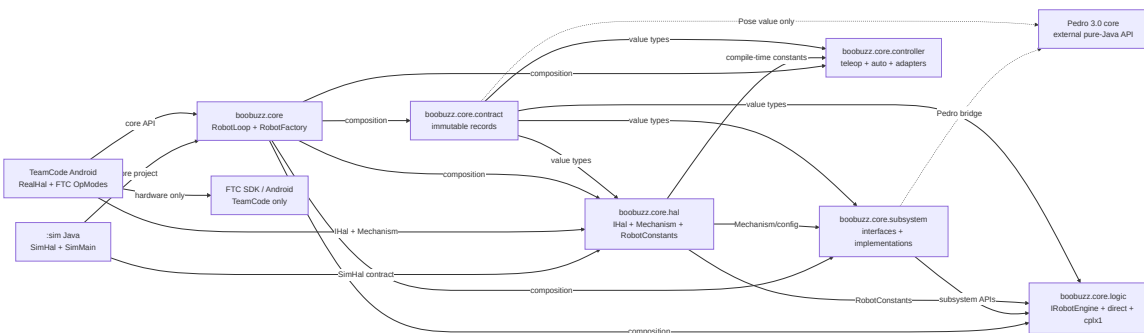
1. Repositories and boundaries

Repository and branch map



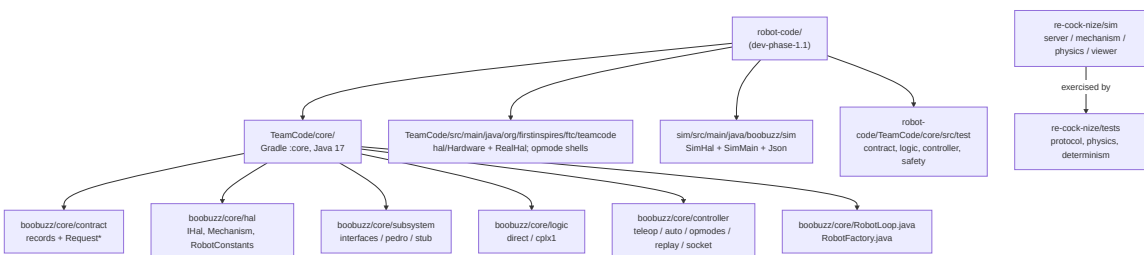
Docs is the record, not a runtime dependency. Java **:core** is shared by the FTC app and Java simulator; Python has no Java-logic import. The old **dev-phase-1** name is historical; the active work is **dev-phase-1.1**.

:core allowed-import graph



Solid arrows are imports or composition; the dotted arrows are the external Pedro boundary. **:core** and **:sim** contain no FTC/Android reference (the SDK guard makes a violation a build error). **DirectMap** calls only the static `cplx1.ShooterLogic.calibratedRpm` helper; no engine state is shared.

Source tree and class locations

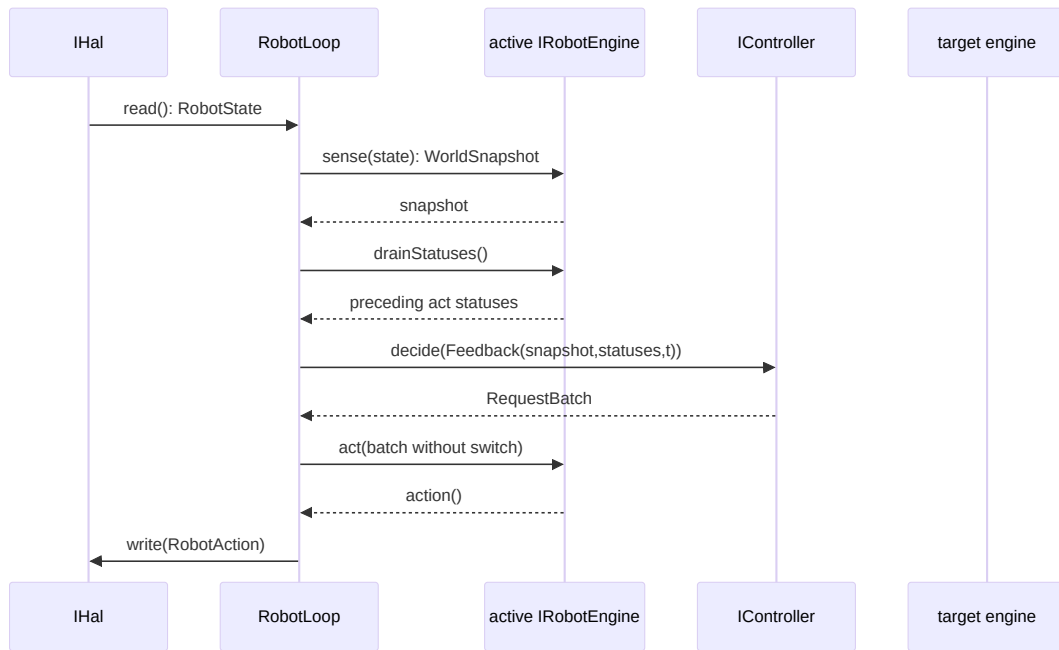


Layer	Package/module path (tree above)	Key classes
Contract	<code>core.contract</code>	<code>RobotState</code> ; <code>Request</code> ; <code>PathRequest</code>
HAL/config	<code>core.hal</code>	<code>IHal</code> , <code>Mechanism</code> , <code>RobotConstants</code>
Subsystems	<code>core.subsystem</code>	<code>IDrive</code> ; <code>IShooter</code> ; <code>ITurret</code>
Pedro bridge	<code>core.subsystem.pedro</code>	<code>PedroDrive</code> ; <code>HalLocalizer</code>
Stubs	<code>core.subsystem.stub</code>	<code>StubShooter</code> ; <code>StubIntake</code>

Layer	Package/module path (tree above)	Key classes
Engines	<code>core.logic.direct / .cplx1</code>	<code>DirectEngine</code> ; <code>CplxEngine1</code>
Controllers	<code>core.controller.{teleop,auto,opmodes,replay,socket}</code>	<code>TeleopController</code> ; <code>AutoController</code>
Orchestration	<code>core</code>	<code>RobotLoop</code> , <code>RobotFactory</code>
FTC shell	<code>teamcode.{hal,opmode}</code> (full package in tree)	<code>Hardware</code> ; <code>RealHal</code> ; <code>*OpMode</code>
Java sim	<code>boobuzz.sim (:sim)</code>	<code>SimHal</code> , <code>SimMain</code> , <code>Json</code>
Python sim/tests	<code>sim / tests</code> (re-cock-nize)	<code>SimServer</code> ; <code>PhysicsBackend</code> ; <code>Viewer</code>

2. Runtime layers

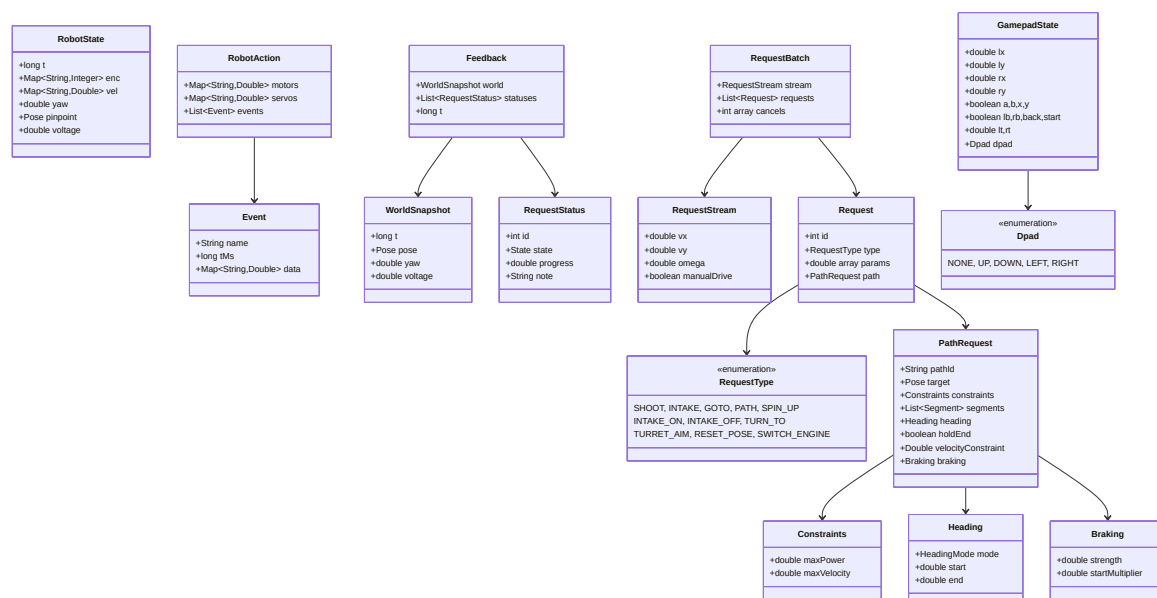
RobotLoop tick sequence



One thread owns the fixed order: read, sense, feedback, decide, optional handoff, act, action, write. Statuses are deliberately one tick delayed; **RobotLoop** never calls the controller from an engine.

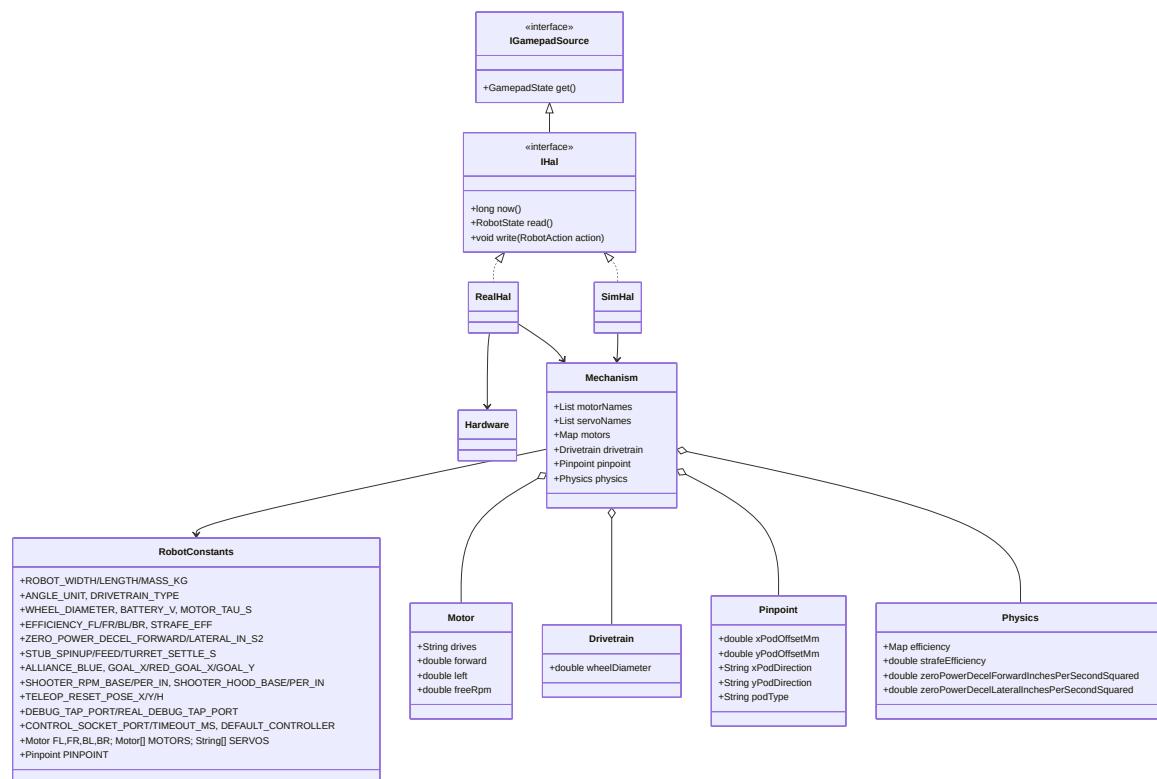
- **HAL** owns the clock, raw sensors, device/socket I/O, and gamepad source; it must not know policy, request jobs, engine internals, or simulator truth.
- **Logic + subsystems** own world projection, arbitration, mechanism state, and statuses; they must not know FTC SDK, gamepad edges, controller callbacks, or wire JSON.
- **Controller** owns policy and sequencing (**Feedback** → **RequestBatch**); it must not know HAL devices, motor names, subsystem objects, or physics.

Contract records and fields



Records copy collections at construction. **RobotState** encoders are integer ticks, velocities ticks/s, Pinpoint inches/radians, yaw radians, and voltage V; **RobotAction** carries named motor powers $[-1,1]$, servo positions $[0,1]$, and events. **truth** from the simulator never enters **RobotState**.

HAL and compile-time configuration



RealHal owns FTC devices; SimHal owns the TCP client and simulator clock. `now()` is a monotonic HAL-clock millisecond value; `read()` returns one `RobotState`, and `write()` applies one complete `RobotAction`. `RobotConstants.java` is the single compile-time configuration source: geometry, motor declarations, Pinpoint, battery, lag, efficiency, deceleration, goals, reset pose, and socket ports. Python `mechanism.py` reads its scalar/string, servo, and one-line motor declarations with regular expressions.

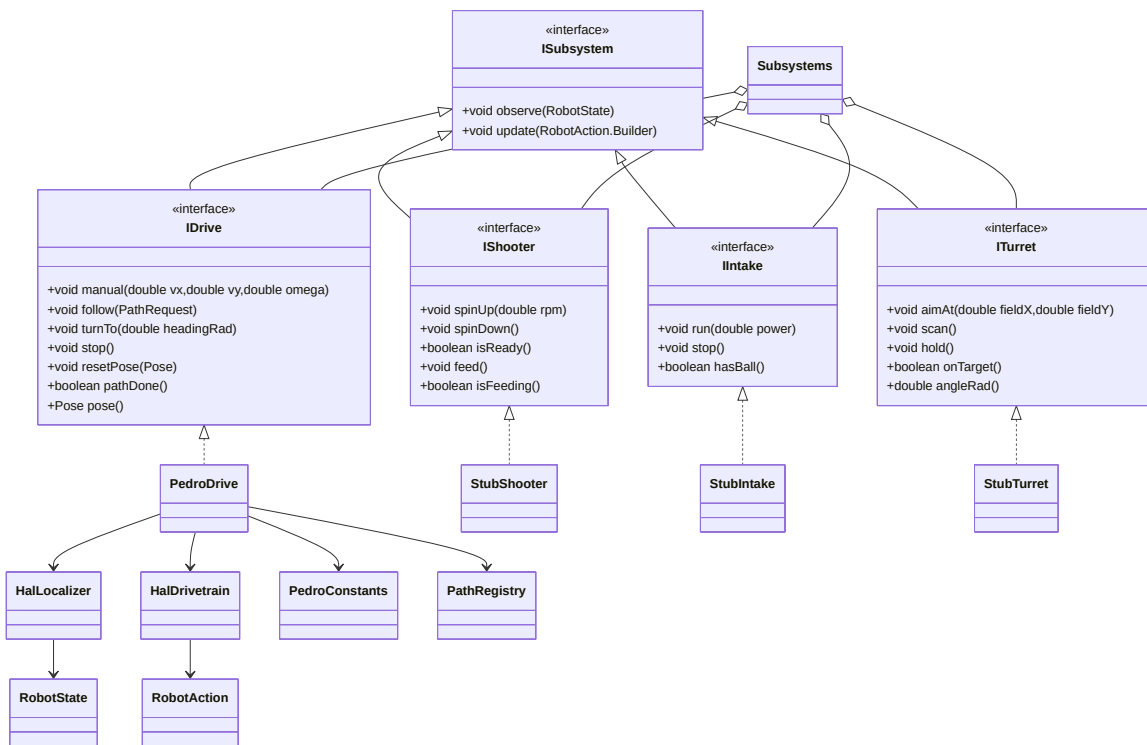
The reader's line-format contract is deliberately narrow (one declaration per physical line):

```
public static final double NAME = value;
public static final String NAME = "value";
public static final Motor FL = new Motor("fl", "wheel", xForward, yLeft, rollerDeg, ticksPerRev, freeRpm);
public static final String[] SERVOS = {};
public static final Pinpoint PINPOINT = new Pinpoint(xOffsetMm, yOffsetMm, xDirection, yDirection, podT);
```

Python requires `ROBOT_WIDTH`, `ROBOT_LENGTH`, `WHEEL_DIAMETER`, `BATTERY_V`, `MOTOR_TAU_S`, `STRAFE_EFF`, and both `ZERO_POWER_DECEL_*` scalars; it defaults missing motor efficiency to 1.0, preserves motor declaration order, and does not parse `PINPOINT` (Java consumes that record). `ROBOT_MASS_KG` is a Java constant but is not yet stored in the Python mechanism model.

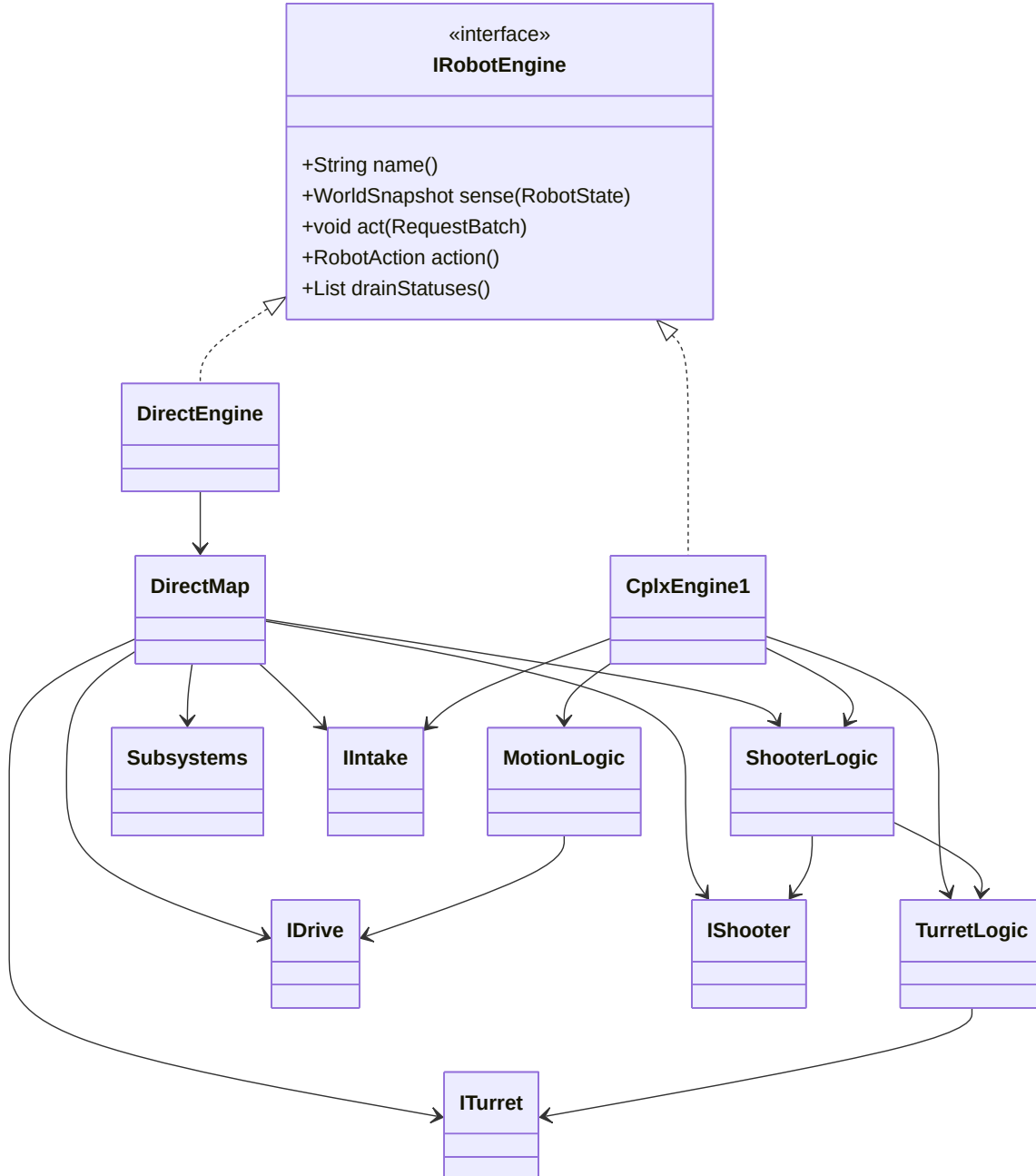
`Intent`, sealed `Drive` variants, `MechanismLoader`, and `mechanism.yaml` are not current interfaces. The YAML decision is closed: compile-time Java constants make configuration errors fail during compilation and remove a runtime parser/moving part.

Subsystem interfaces, Pedro bridge, and stubs



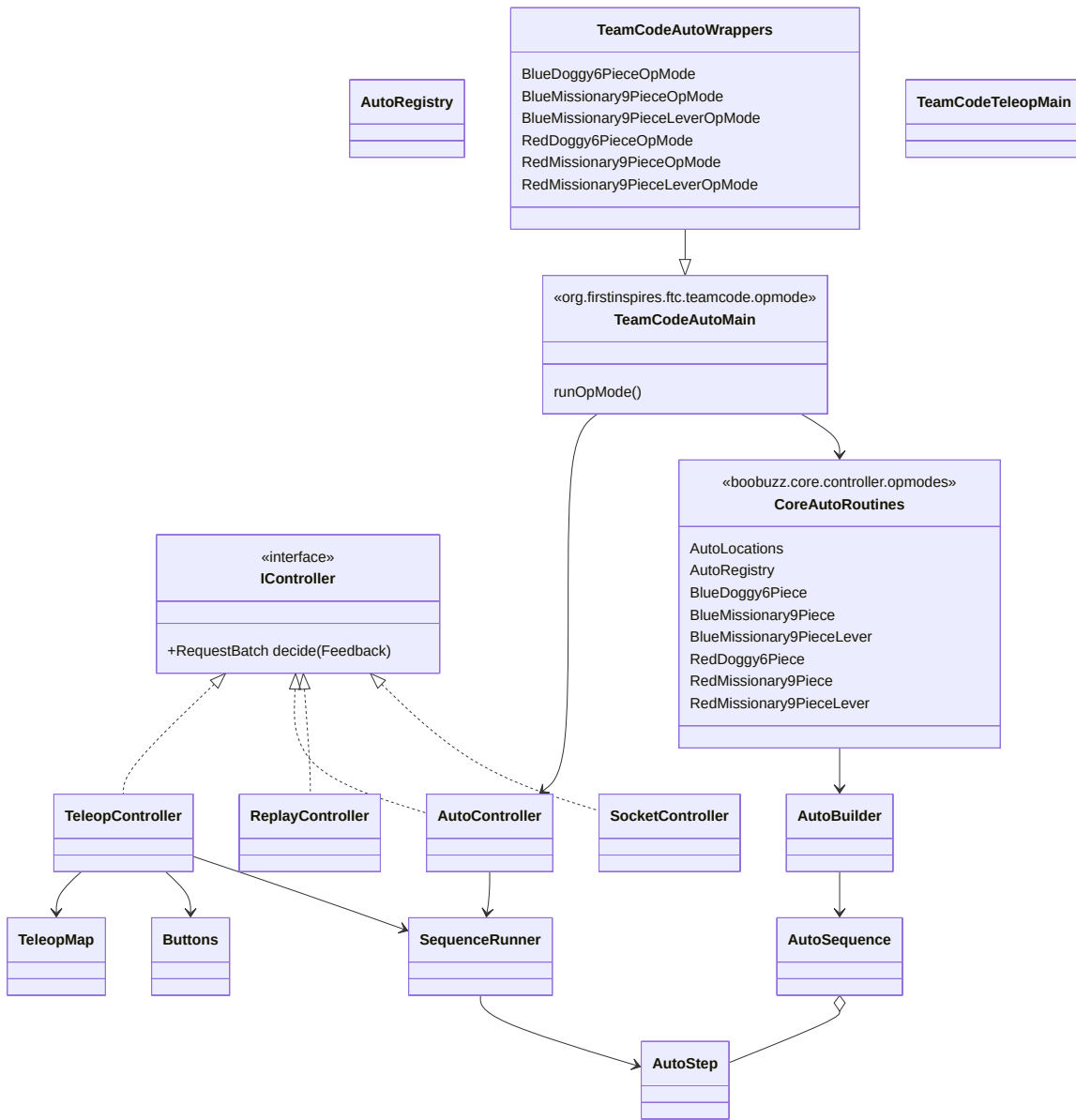
`Subsystems.observe` samples all four mechanisms, then `update` assembles one action in drive/shooter/intake/turret order. Pedro classes translate the contract to Pedro 3.0; stubs provide timing/events until real mechanisms land. To add one: define a narrow `I... extends ISubsystem`, add a stub/real implementation, add its field and fixed-order calls to `Subsystems`, construct it in `RobotFactory`, then map requests in the owning engine and test it.

Logic engines and modules



DirectEngine owns wiring, stream arbitration, and two direct jobs. **CplxEngine1** delegates motion, automatic turret tracking, shooter sequencing, and intake; both engines share one **Subsystems** instance. **DirectMap** uses only the static calibrated-RPM helper from **ShooterLogic**; job state is not shared. The drive surface today is Manual/Velocity (the level stream), GoTo (**GOTO**), FollowPath (**PATH**), TurnTo, and Hold (no active request or **holdEnd**).

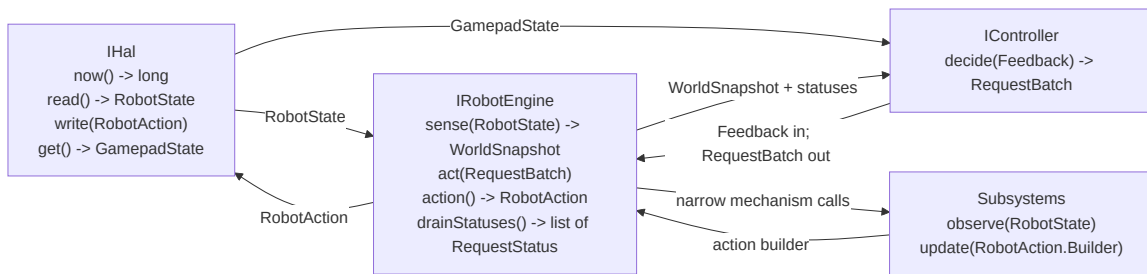
Controllers, FTC opmodes, replay, and socket input



Controllers see **Feedback** and return **RequestBatch**; they do not see HAL or subsystems. **TeleopController** can run a Y-started sequence in the background; **TeamCodeAutoMain** supplies the six registry-backed FTC entry points. Replay consumes JSONL seam batches; **SocketController** exchanges feedback and batches off-thread, and emits one **CANCEL_ALL** after its receive timeout.

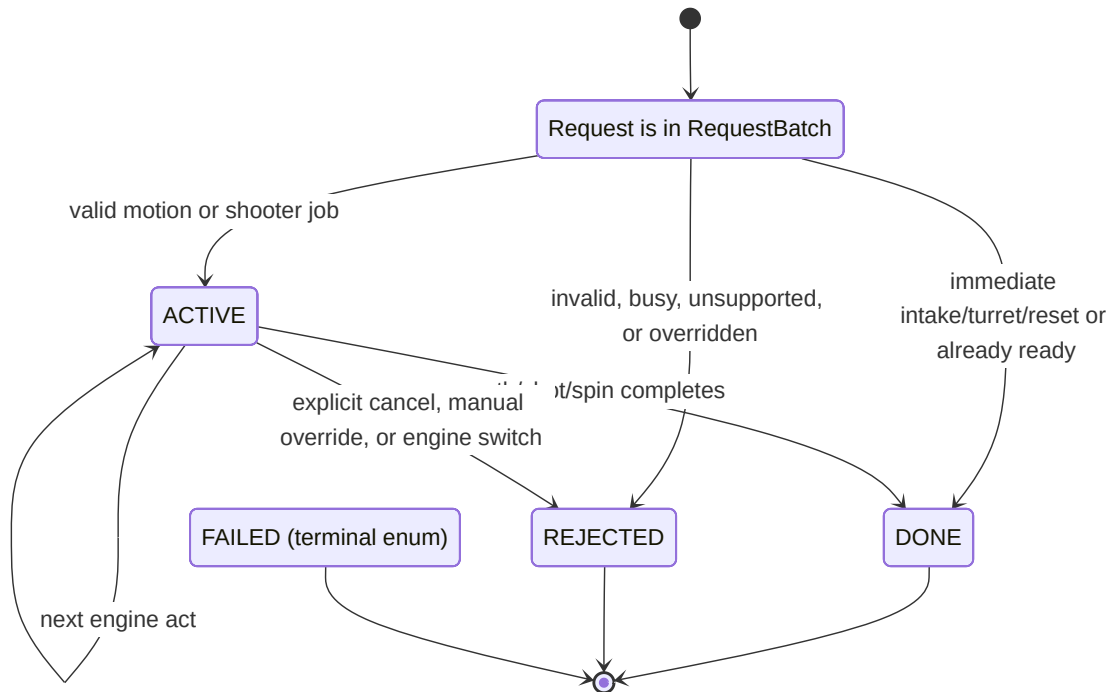
3. Contracts and control flow

Inter-module seam (allowed directions)



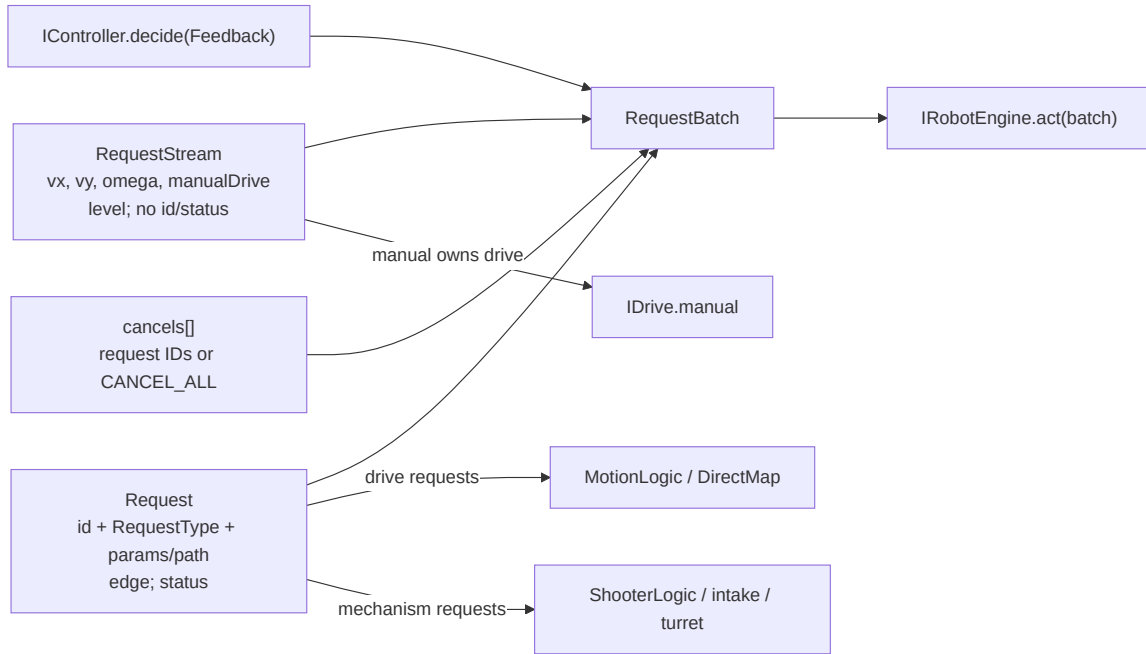
Only RobotLoop calls the arrows in order; controllers never call a subsystem, and engines never open the JSON socket. RobotConstants/Mechanism are construction data, not a mutable cross-module bus.

RequestStatus lifecycle



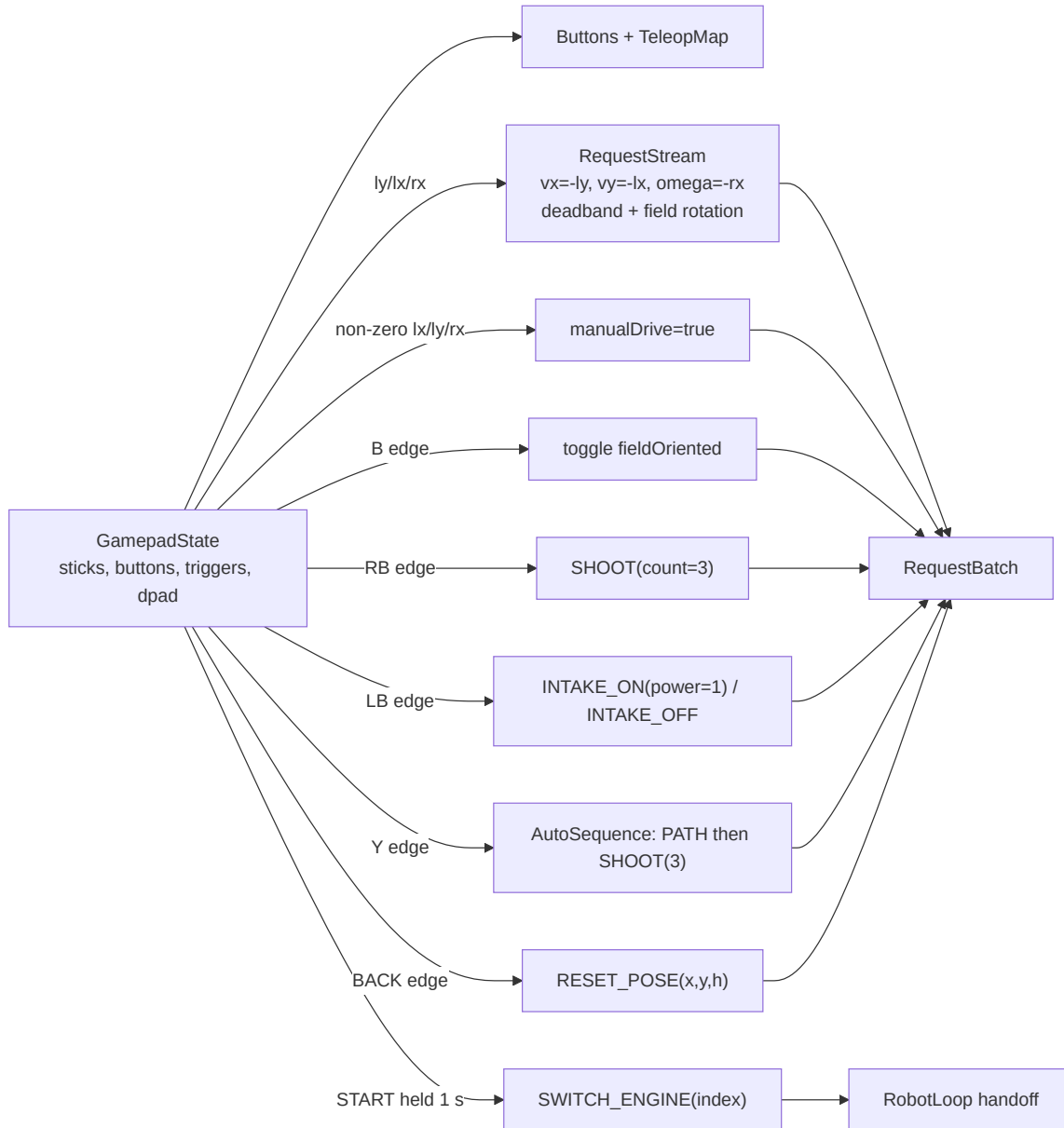
RequestStatus fields are (id,state,progress,note); terminal states are DONE, FAILED, and REJECTED. The controller receives statuses on the next tick through Feedback; current code emits ACTIVE, DONE, and REJECTED.

Level stream versus edge request



RequestStream is present every tick, including idle; **Request** objects are edge-triggered and immutable. Manual stream input cancels an active drive and rejects a simultaneous drive request; **CANCEL_ALL** is reserved for handoff.

Teleop gamepad to request map

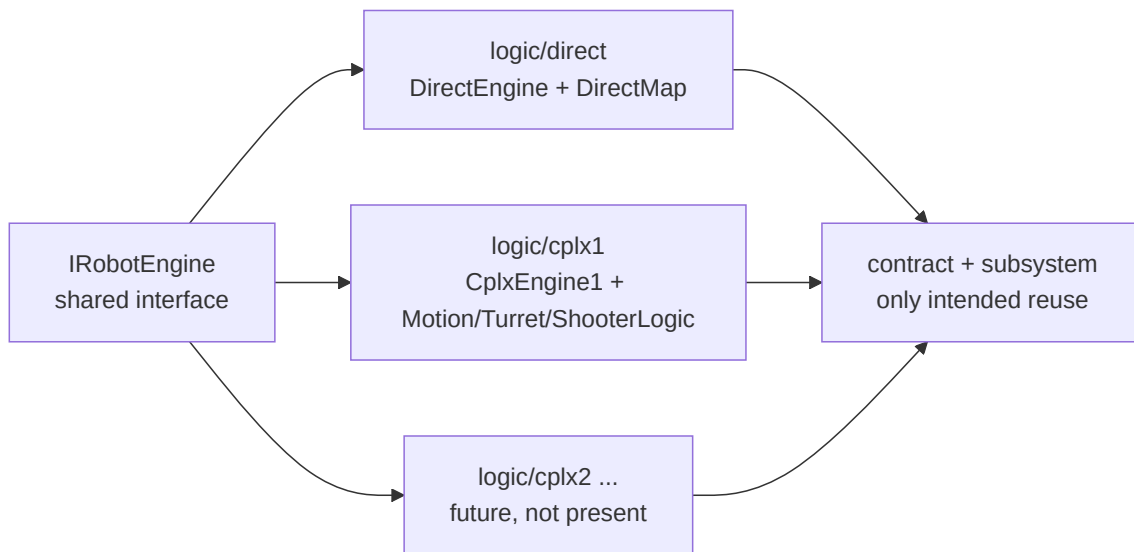


The map starts field-oriented, applies a 0.05 deadband, and allocates teleop IDs from 10,000. A Y sequence moves +12 inches at its current heading then shoots three; a real stick sample aborts it. A/B/X/R/Y/LT/RT/D-pad are currently unmapped.

Shared state is intentionally small: immutable static **RobotConstants**, the immutable **Mechanism.DEFAULT**, and per-loop engine jobs/statuses plus private subsystem state. Python keeps per-backend pose/RNG/logs. There is no singleton engine, global request queue, or shared truth pose; engine switching reuses the same **Subsystems** object.

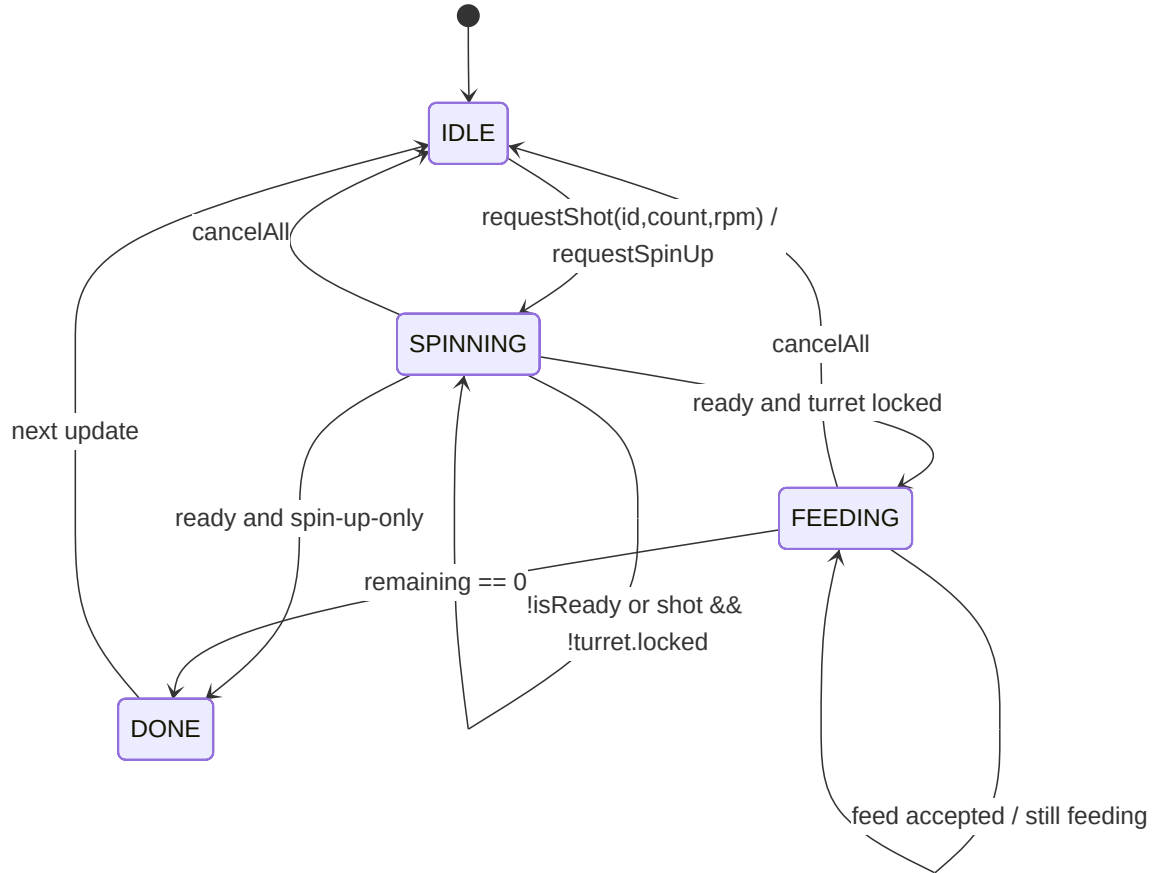
4. Logic state machines

`cplx_engine_N` siblings



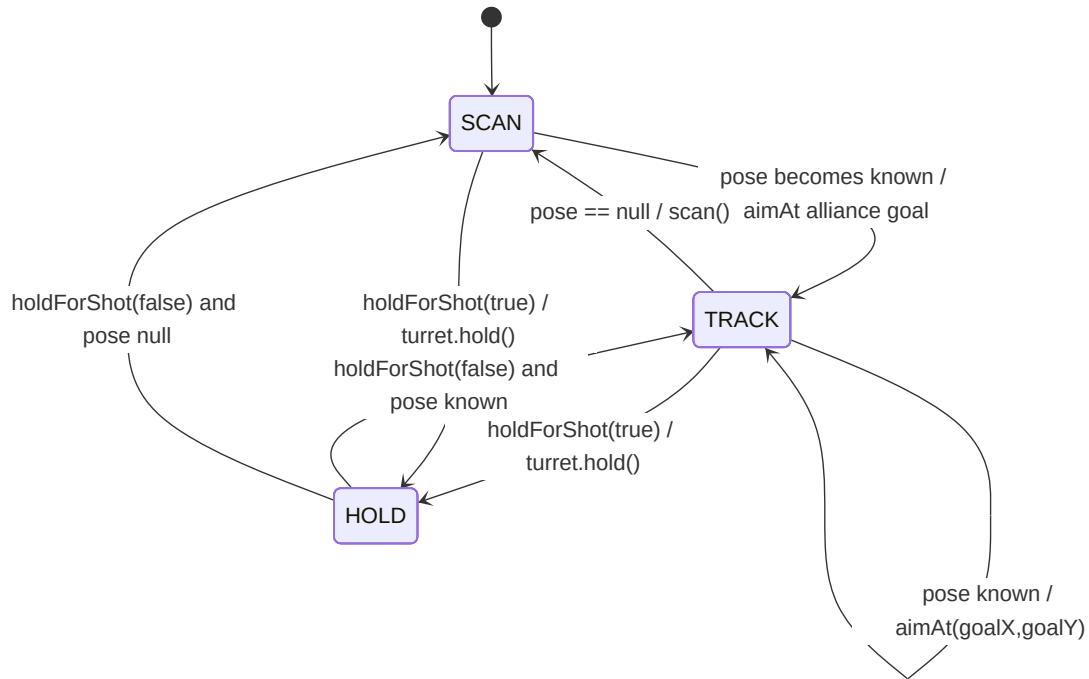
Each `cplx_engine_N` is a sibling implementation folder; siblings do not import one another or share engine state. The canonical current name is `cplx1`; the historical `cplx_engine_1` string remains an input alias in `RobotFactory`.

ShooterLogic



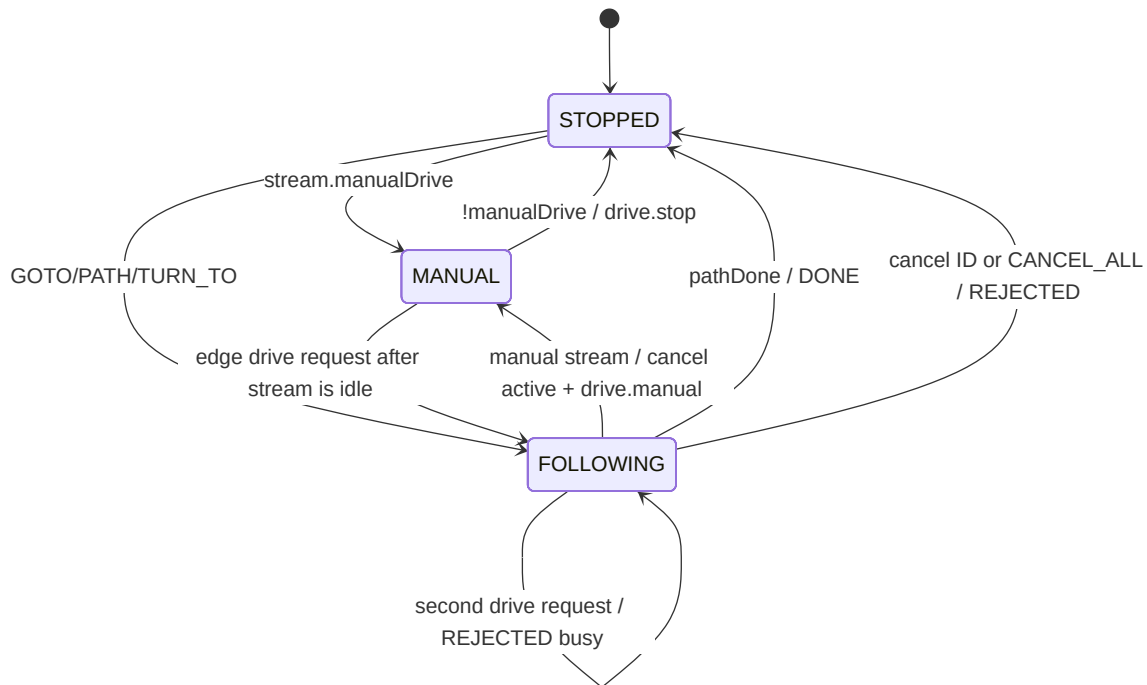
CplxEngine1 validates count/RPM, then **ShooterLogic** owns aiming, RPM/hood selection, feed count, and statuses. The direct engine uses the same count-only calibrated RPM helper when no RPM parameter is supplied.

TurretLogic track, scan, and hold



There is no public enum: `holdingForShot` and `targetKnown` are the state. The goal is `(GOAL_X,GOAL_Y)` for blue or `(RED_GOAL_X,GOAL_Y)` for red; controllers never issue turret commands directly.

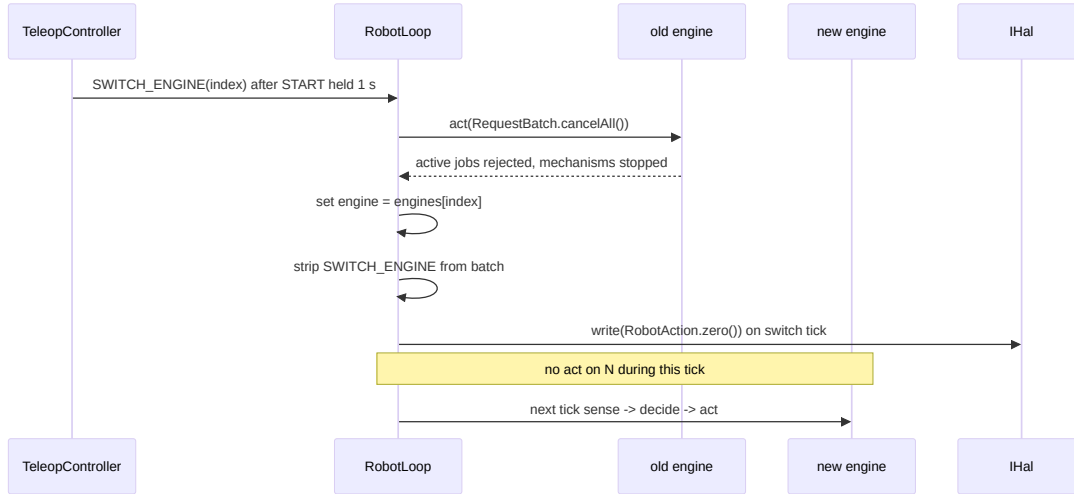
MotionLogic drive arbitration



DirectEngine performs the same arbitration around `DirectMap`; both paths give manual input priority.

Motion requests validate their payload before creating an active job, and **RESET_POSE** cancels active motion before resetting the software/localizer pose.

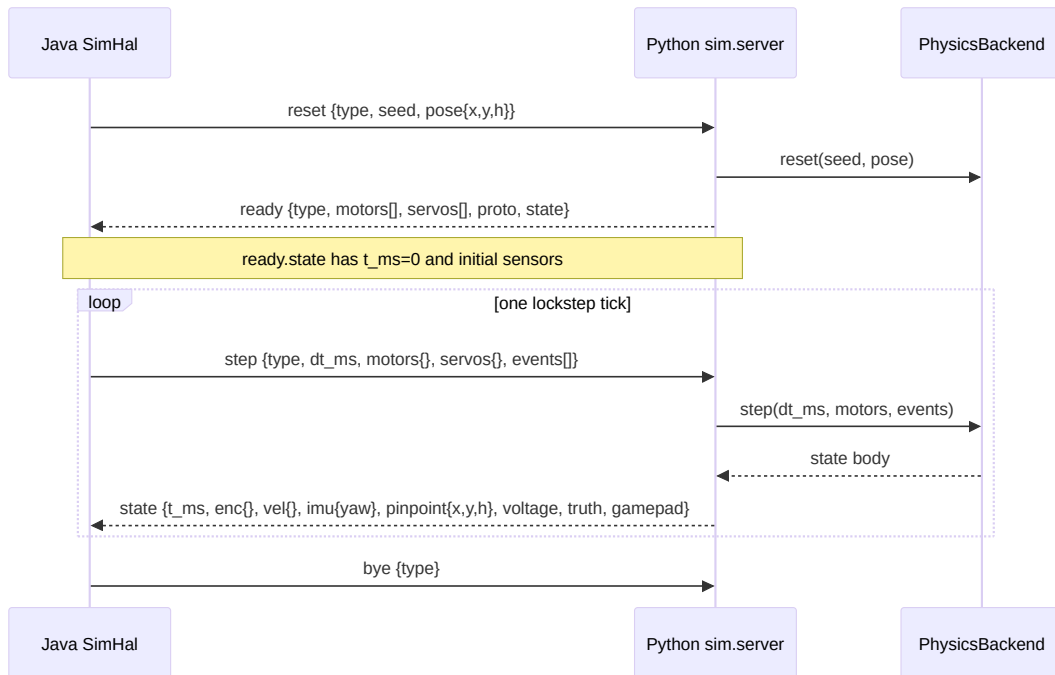
Engine switch



RobotFactory orders index 0 = `DirectEngine`, index 1 = `CplxEngine1` over one shared `Subsystems` object. The loop, not either engine, consumes the switch request.

5. Simulator

SimHal and Python server protocol

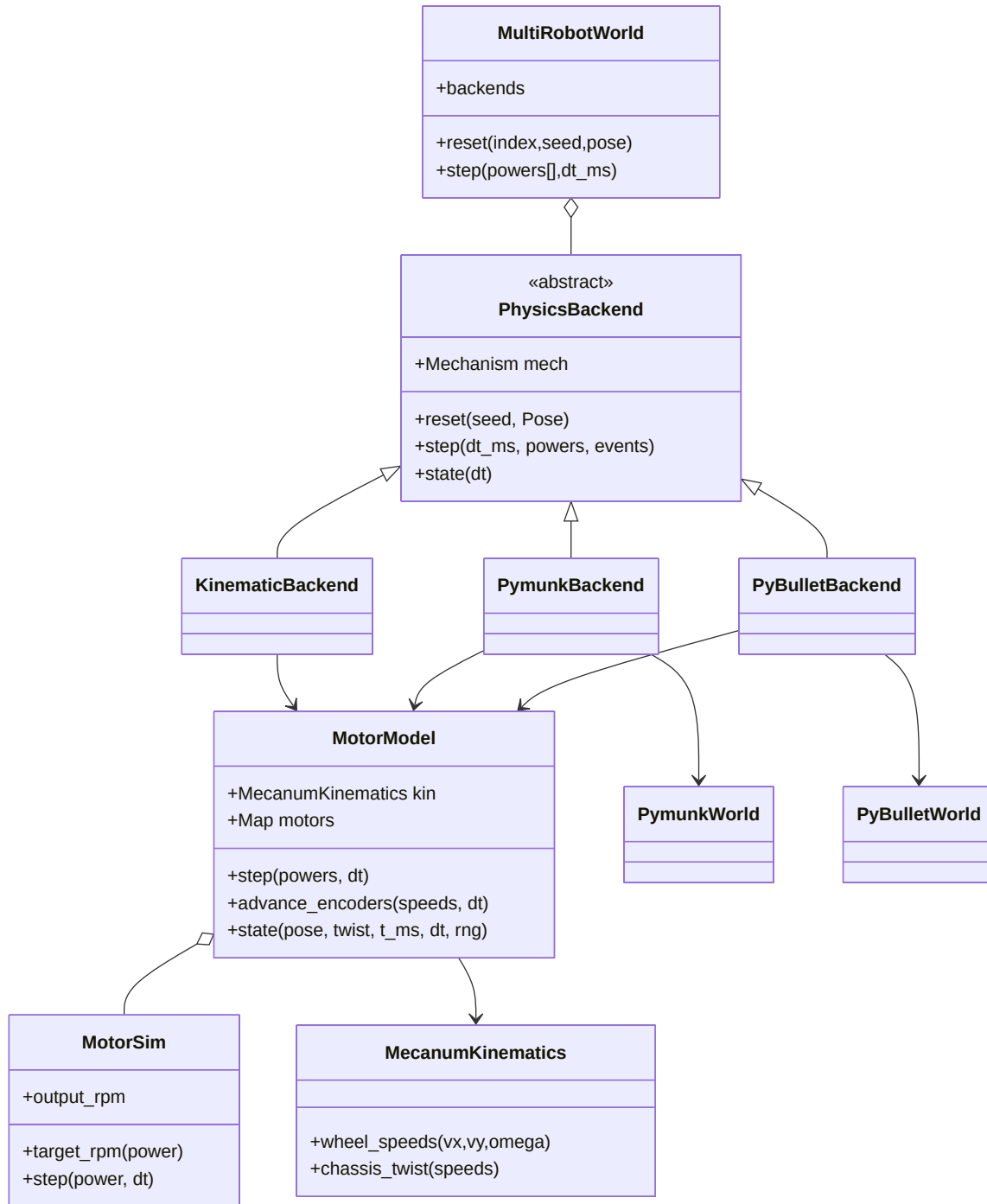


TCP is line-delimited JSON on `127.0.0.1:5555`; Java supplies positive `dt_ms`, so physics advances exactly once per `step`. `events[]` entries are `{name,t_ms,data{}}`; `truth` is viewer/test-only and is retained only

in `SimHal.truth`, not copied into `RobotState`. Multi-robot mode uses one port per robot and the same message shapes.

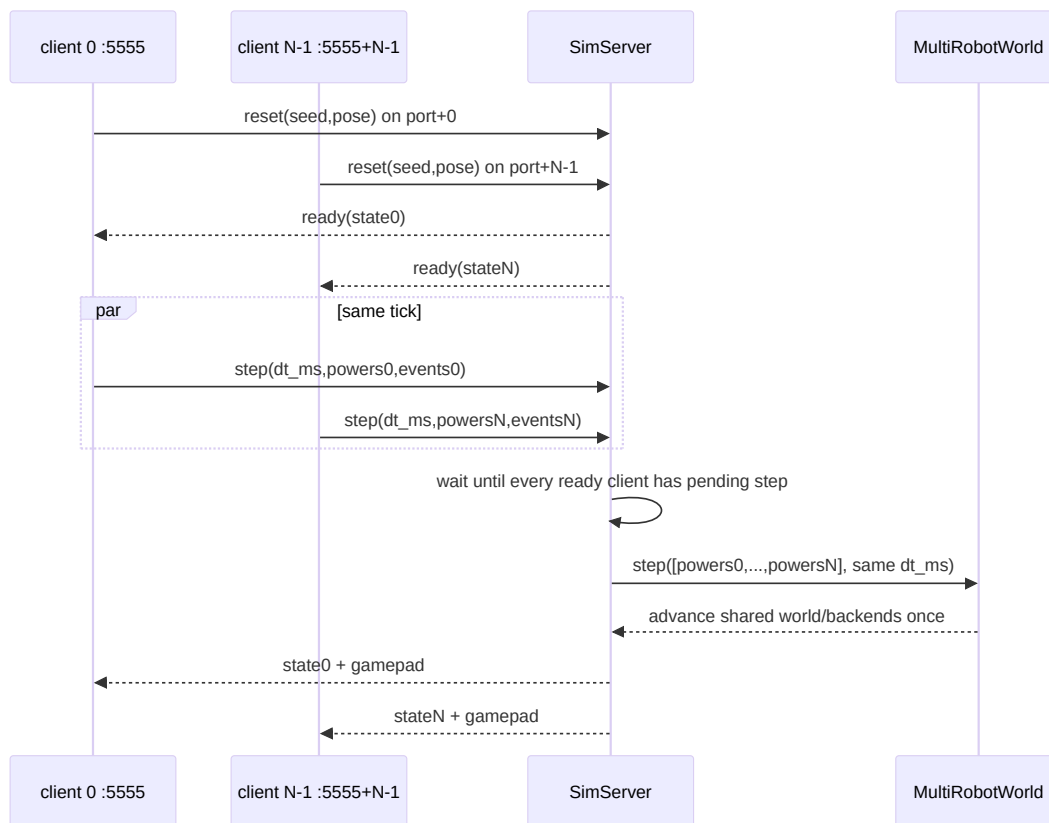
Message	Fields on the wire
Java -> Python <code>reset</code>	<code>type</code> , <code>seed</code> , <code>pose{x,y,h}</code>
Python -> Java <code>ready</code>	<code>type</code> , <code>motors[]</code> , <code>servos[]</code> , <code>proto</code> , <code>state</code>
Java -> Python <code>step</code>	<code>type</code> , <code>positive dt_ms</code> , <code>motors{}</code> , <code>servos{}</code> , <code>events[]</code>
Python -> Java <code>state</code>	<code>type</code> , <code>t_ms</code> , <code>enc{}</code> , <code>vel{}</code> , <code>imu{yaw}</code> , <code>pinpoint{x,y,h}</code> , <code>voltage</code> , <code>truth{x,y,h}</code> , <code>gamepad{lx,ly}</code>
Java -> Python <code>bye</code>	<code>type</code>

Python physics backend classes



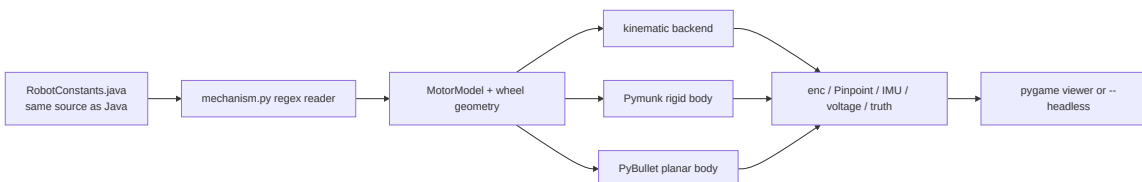
All backends share **MotorSim** (power clamp, calibrated free RPM, first-order lag `MOTOR_TAU_S=0.1 s`), roller-angle mecanum kinematics, integer encoders, and seeded noisy Pinpoint/IMU state. Pymunk is the default rigid body; PyBullet is optional; kinematic is the Euler comparison baseline.

Multi-robot lockstep (--robots N)



The server rejects mismatched `dt_ms`, advances only after all connected/ready slots in the current reset epoch submit, and evicts silent clients after the lockstep deadline. Resetting one body does not reset the others; a late client starts a new reset barrier. Pymunk and PyBullet share one world; kinematic backends step independently in the same barrier.

Simulator capabilities and calibration



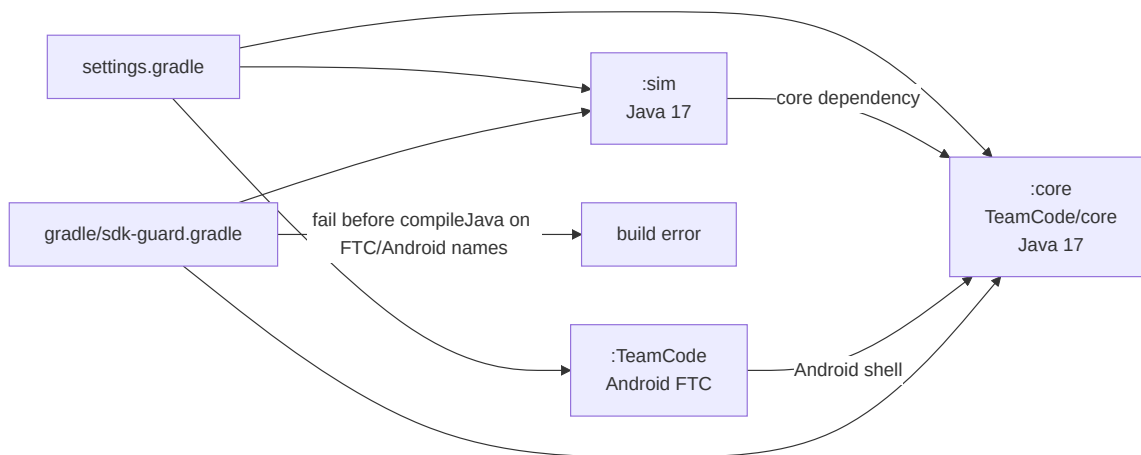
The field is 144 x 144 inches; pose and Pinpoint are inches/radians. Current calibration is `STRAFE_EFF=0.7346`, forward/lateral zero-power deceleration `36.17/85.98 in/s2`, battery 12 V, wheel roller angles +/-45 degrees, and wheel positions +/-6.5 forward and +/-5.5 left. The model does not simulate shooter/intake/turret physics, voltage sag, current draw, or back-EMF; Java stubs emit mechanism timing events instead.

Calibration	Value	Origin
ROBOT_WIDTH, ROBOT_LENGTH	18.0 in, 18.0 in	current chassis cons
ROBOT_MASS_KG	12.0 kg	Java constant; rigid
WHEEL_DIAMETER	4.0 in	current wheel geom
BATTERY_V, MOTOR_TAU_S	12.0 V, 0.1 s	compile-time simula

Calibration	Value	Origin
STRAFE_EFF	0.7346	last season yVelocity
zero-power decel (forward/lateral)	36.17 / 85.98 in/s ²	last season's Constants
roller angles	+45 deg / -45 deg	RobotConstants m
wheel positions	+/-6.5 forward, +/-5.5 left in	RobotConstants m
ticks/rev; free RPM	537.7; 351.55735379568756	declarations; RPM
wheel efficiency FL/FR/BL/BR	1.0 each	placeholder, not me
Pinpoint	(161.0, 0.0) mm; FORWARD/REVERSED; goBILDA_4_BAR_POD	last season's archi
stub timing	spin-up 0.5 s; feed 0.2 s; turret settle 0.3 s	Java timing-only st

6. Build and run

Gradle and SDK guard



`:core` is a Java library with Pedro 3.0 core; `:sim` is an application and never depends on `:TeamCode`. The guard scans main/test sources for `com.qualcomm`, `org.firstinspires`, or `android`. before compilation.

```

export JAVA_HOME=/usr/lib/jvm/java-21-openjdk
# from robot-code/
./gradlew :core:test :sim:test
./gradlew :sim:run --args="--headless --steps 500 --dt 20"
cd ../re-cock-nize
python -m sim.server --headless --physics pymunk
cd ../robot-code
./gradlew :sim:run --args="--controller replay --bag run.jsonl --steps 500"

```

`architecture/build.sh` uses installed `mmmc` plus Chrome to emit labeled SVG and tight PDF diagrams, then invokes Pandoc with a real TeX engine. No package manager or browser download is part of the build.

Appendix B -- open decisions

- Freeze the Request/Feedback contract after the current R3/R4 seam review.
- Confirm Pinpoint offsets: 161/0 mm in `RobotConstants` versus the proposed -84/-168 mm measurement.
- Resolve Android Studio synchronization for nested Gradle project `:core`.
- Fold duplicate `RobotConstants.Motor/Pinpoint` and `Mechanism.Motor/Pinpoint` records in the next cleanup.

- Define whether `RequestStatus.FAILED` should gain an engine emitter; current engines reject invalid/cancelled work but do not emit `FAILED`.

Appendix C -- methodology and limits

- Source-of-truth is the checked-out Java/Python code; diagrams do not invent behavior for unimplemented seams.
- Documentation work uses hierarchical agents coordinated through `bp`, as described in the repository README; each claim is checked against source.
- `protokol.md` and design decisions remain owned by `ftc-main`; this document records the implementation boundary and current open choices.